

LAPORAN PROJECT ALGORITMA PEMROGRAMAN

*“Simulation Feed Water Tank Monitoring & Control
System Berbasis GUI dengan Integrasi BPCS dan SIS”*



Dosen:

Ahmad Radhy, S.Si., M.Si.

Oleh:

Fauzi Abdul Rozaq 2042241017

Sintia Ompusunggu 2042241113

DEPARTEMEN TEKNIK INSTRUMENTASI
FAKULTAS VOKASI
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
2026

Daftar Isi

1	Pendahuluan	2
1.1	Latar Belakang	2
1.2	Rumusan Masalah	2
1.3	Tujuan Project	2
1.4	Manfaat Project	3
2	Tinjauan Pustaka	3
2.1	Feed Water Tank (FWT)	3
2.2	BPCS dan SIS	3
2.3	Pemrosesan Sinyal: <i>Noise</i> dan Filter <i>Moving Average</i>	3
2.4	Arsitektur GUI <i>Immediate Mode</i> dengan <i>eframe</i>	4
3	Metode Penelitian	4
3.1	Struktur Project	4
3.2	Arsitektur Komponen GUI	5
3.3	Desain Algoritma	5
3.3.1	Pemrosesan Sinyal: <i>Noise</i> + Filter MA	5
3.3.2	Lapisan BPCS: Kontrol Proporsional	6
3.3.3	Lapisan SIS: <i>Interlock</i> SDV	6
3.4	Flowchart Sistem	6
4	Implementasi Program	7
4.1	Inisialisasi Aplikasi dan Pemuatan Gambar Latar	7
4.2	Struktur Data Utama	8
4.3	Pemrosesan Sinyal dan Logika Kontrol	8
4.4	Visualisasi Status Komponen	9
5	Hasil dan Pembahasan	9
5.1	Tampilan GUI	9
5.2	Pengujian Filter <i>Moving Average</i>	9
5.3	Validasi Logika BPCS	9
5.4	Validasi Interlock SIS	9
6	Penutup	10
6.1	Kesimpulan	10
6.2	Saran	10

1 Pendahuluan

1.1 Latar Belakang

Keandalan sistem pasokan air pada *boiler* di industri sangat bergantung pada efektivitas pemantauan *Feed Water Tank* (FWT). FWT berfungsi sebagai reservoir utama air pengisi *boiler*, sehingga ketidakstabilan level di dalamnya dapat menyebabkan gangguan produksi hingga kerusakan peralatan yang signifikan.

Pada project ini dikembangkan aplikasi simulasi bernama **Simulation Feed Water Tank** menggunakan bahasa pemrograman Rust. Aplikasi menampilkan antarmuka grafis (GUI) berbasis *eframe* versi 0.28.1 yang mampu memvisualisasikan kondisi proses secara *real-time*. Latar belakang antarmuka menggunakan gambar P&ID (*Piping and Instrumentation Diagram*) yang dimuat menggunakan crate *egui_extras* dengan fitur *image loader*, sehingga tampilan menyerupai sistem SCADA industri nyata.

Simulasi ini mengintegrasikan dua lapisan kontrol: *Basic Process Control System* (BPCS) untuk menjaga level pada *setpoint* normal, dan *Safety Instrumented System* (SIS) sebagai lapisan perlindungan terakhir yang secara independen mengoperasikan *Shutdown Valve* (SDV) inlet dan outlet.

1.2 Rumusan Masalah

1. Bagaimana membangun antarmuka GUI berbasis *eframe* yang menampilkan visualisasi P&ID FWT secara *real-time*?
2. Bagaimana mengimplementasikan pengolahan sinyal digital (*noise* dan *moving average filter*) untuk mensimulasikan karakteristik sensor Level Transmitter (LT) di lapangan?
3. Bagaimana mengintegrasikan logika BPCS dan SIS secara terpisah agar kestabilan proses terjaga sekaligus kondisi kritis (*overflow* dan *dry run*) dapat dicegah?

1.3 Tujuan Project

1. Membangun aplikasi simulasi FWT berbasis GUI menggunakan crate *eframe* 0.28.1 di bahasa pemrograman Rust.
2. Memuat dan menampilkan gambar latar P&ID menggunakan *egui_extras* dengan fitur *all_loaders*.
3. Menerapkan algoritma *moving average* untuk memproses sinyal sensor yang mengandung *random noise* dari crate *rand* 0.8.
4. Mengimplementasikan kontrol proporsional pada lapisan BPCS dan logika *interlock* dua-kondisi pada lapisan SIS.

1.4 Manfaat Project

Project ini memberikan pengalaman langsung dalam merancang sistem kontrol berlapis sesuai standar otomasi industri, sekaligus mendalami pemrograman sistem dengan Rust mencakup manajemen memori, rendering GUI *immediate mode*, dan pemrosesan sinyal sederhana.

2 Tinjauan Pustaka

2.1 Feed Water Tank (FWT)

Feed Water Tank adalah komponen kritis pada sistem pembangkit uap industri yang berfungsi sebagai reservoir air pengisi *boiler*. FWT umumnya dilengkapi fungsi deaerasi untuk menghilangkan oksigen terlarut yang dapat menyebabkan korosi. Pemantauan level FWT secara kontinu sangat penting untuk mencegah dua kondisi berbahaya:

- **Overflow** (level $\geq 90\%$): aliran masuk melebihi kapasitas tangki, berpotensi merusak peralatan *downstream*.
- **Dry Run** (level $\leq 10\%$): level terlalu rendah menyebabkan kavitasi pada pompa pengisi *boiler*.

2.2 BPCS dan SIS

Standar IEC 61511 mendefinisikan dua lapisan kontrol terpisah pada sistem proses industri:

- **BPCS (*Basic Process Control System*)**: menjaga variabel proses pada *setpoint* di kondisi operasi normal melalui kontrol reguler.
- **SIS (*Safety Instrumented System*)**: lapisan perlindungan independen yang membawa sistem ke *safe state* ketika BPCS gagal atau kondisi berbahaya terdeteksi.

Kedua sistem harus beroperasi secara terpisah—baik dari sisi perangkat keras maupun logika perangkat lunak—agar kegagalan BPCS tidak ikut melumpuhkan SIS.

2.3 Pemrosesan Sinyal: *Noise* dan Filter *Moving Average*

Sensor di lapangan terpapar interferensi elektromagnetik yang menghasilkan *noise* acak pada sinyal keluarannya. Untuk mensimulasikan kondisi nyata, *noise* acak ditambahkan pada nilai input dasar menggunakan crate *rand*, kemudian sinyal diperhalus dengan filter *Moving Average* (MA):

$$PV_{\text{filtered}} = \frac{1}{N} \sum_{i=0}^{N-1} (x_{\text{raw}} + \eta)_{t-i} \quad (1)$$

di mana N adalah panjang jendela filter (dalam project ini $N = 5$), x_{raw} adalah nilai input dari *slider* operator, dan η adalah *noise* acak yang dibangkitkan setiap siklus.

2.4 Arsitektur GUI *Immediate Mode* dengan *eframe*

Crate *eframe* 0.28.1 menggunakan paradigma *immediate mode GUI*, di mana seluruh tampilan dirender ulang dari awal pada setiap *frame*. Pendekatan ini cocok untuk aplikasi monitoring data sensor karena setiap pembaruan nilai langsung tercermin pada layar tanpa perlu manajemen *state* widget yang kompleks.

Untuk memuat gambar latar (berkas `background.png` berformat P&ID), project ini menggunakan *egui_extras* dengan fitur `all_loaders` yang mendukung format PNG melalui crate *image* 0.24.

3 Metode Penelitian

3.1 Struktur Project

Project dikembangkan sebagai paket Cargo bernama `Simulation_Feed_Water_Tank` dengan konfigurasi berikut pada `Cargo.toml`:

Listing 1: Konfigurasi Cargo.toml

```

1 [package]
2 name      = "Simulation_Feed_Water_Tank"
3 version   = "0.1.0"
4 edition   = "2024"
5
6 [dependencies]
7 eframe     = "0.28.1"
8 egui_extras = { version = "0.28.1", features = ["all_loaders"] }
9 image      = { version = "0.24",    features = ["png"] }
10 rand       = "0.8"

```

Setiap crate memiliki peran spesifik:

- **eframe 0.28.1:** *framework* utama untuk jendela aplikasi dan siklus render GUI.
- **egui_extras 0.28.1 (all_loaders):** ekstensi *egui* untuk memuat dan merender gambar (PNG) sebagai latar P&ID.
- **image 0.24 (png):** *decoder* PNG yang digunakan secara internal oleh *egui_extras*.
- **rand 0.8:** pembangkit bilangan acak untuk mensimulasikan *noise* sensor.

3.2 Arsitektur Komponen GUI

Tampilan GUI seperti terlihat pada Gambar 1 terdiri atas komponen-komponen berikut yang dirender di atas gambar latar P&ID:

- **Label 100%/0%:** penanda skala level tangki (atas dan bawah).
- **Blok SDV Inlet:** indikator status *Shutdown Valve* pada sisi masuk.
- **Blok SDV Outlet:** indikator status *Shutdown Valve* pada sisi keluar.
- **Blok PV:** menampilkan nilai *Process Variable* (level tangki terfilter).
- **Blok SP 50%:** *setpoint* default referensi BPCS.
- **Indikator Pump:** menunjukkan status operasi pompa.
- **Indikator LT:** simbol *Level Transmitter* yang menghubungkan tangki ke sistem kontrol.

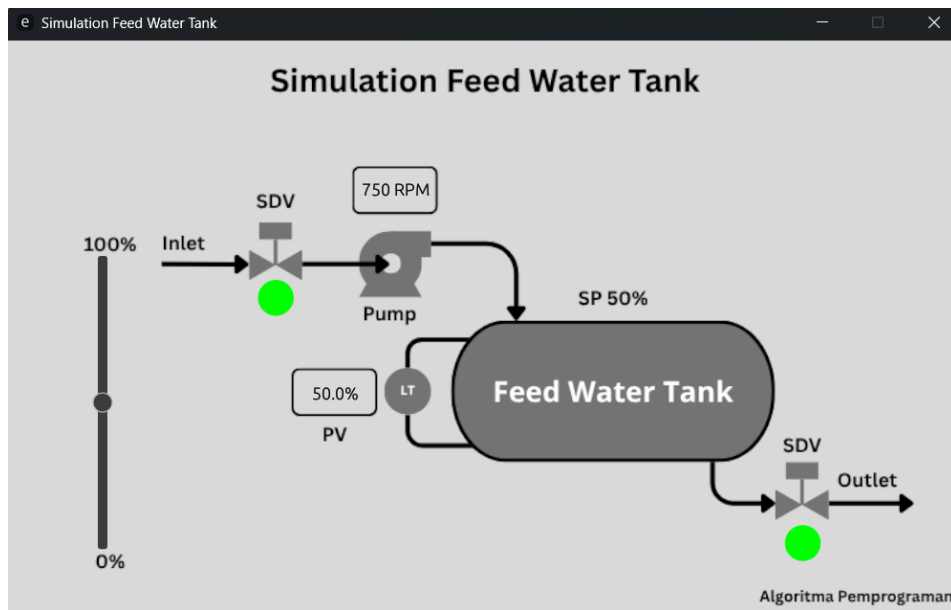


Figure 1: Tampilan GUI Simulation Feed Water Tank

3.3 Desain Algoritma

3.3.1 Pemrosesan Sinyal: *Noise* + Filter MA

Setiap siklus render, sinyal mentah dari input operator ditambahkan *noise* acak menggunakan *rand*, kemudian nilai tersebut dimasukkan ke *buffer* riwayat berukuran $N = 5$. Nilai PV terfilter dihitung sebagai rata-rata dari semua sampel dalam *buffer*.

3.3.2 Lapisan BPCS: Kontrol Proporsional

BPCS menghitung sinyal kontrol berdasarkan selisih antara *setpoint* ($SP = 50\%$) dan nilai PV terfilter:

$$CV_{BPCS} = K_p \times (SP - PV_{\text{filtered}}) \quad (2)$$

Nilai CV_{BPCS} menentukan seberapa jauh katup kontrol dibuka untuk mengoreksi level tangki menuju *setpoint*.

3.3.3 Lapisan SIS: *Interlock* SDV

SIS beroperasi sepenuhnya independen dari BPCS dengan dua kondisi *interlock*:

- Jika $PV_{\text{filtered}} \geq 90\%$: **Inlet SDV ditutup** (mencegah *overflow*).
- Jika $PV_{\text{filtered}} \leq 10\%$: **Outlet SDV ditutup dan pompa dimatikan** (mencegah *dry run*).

SIS tidak dapat di-*override* oleh operator maupun BPCS.

3.4 Flowchart Sistem

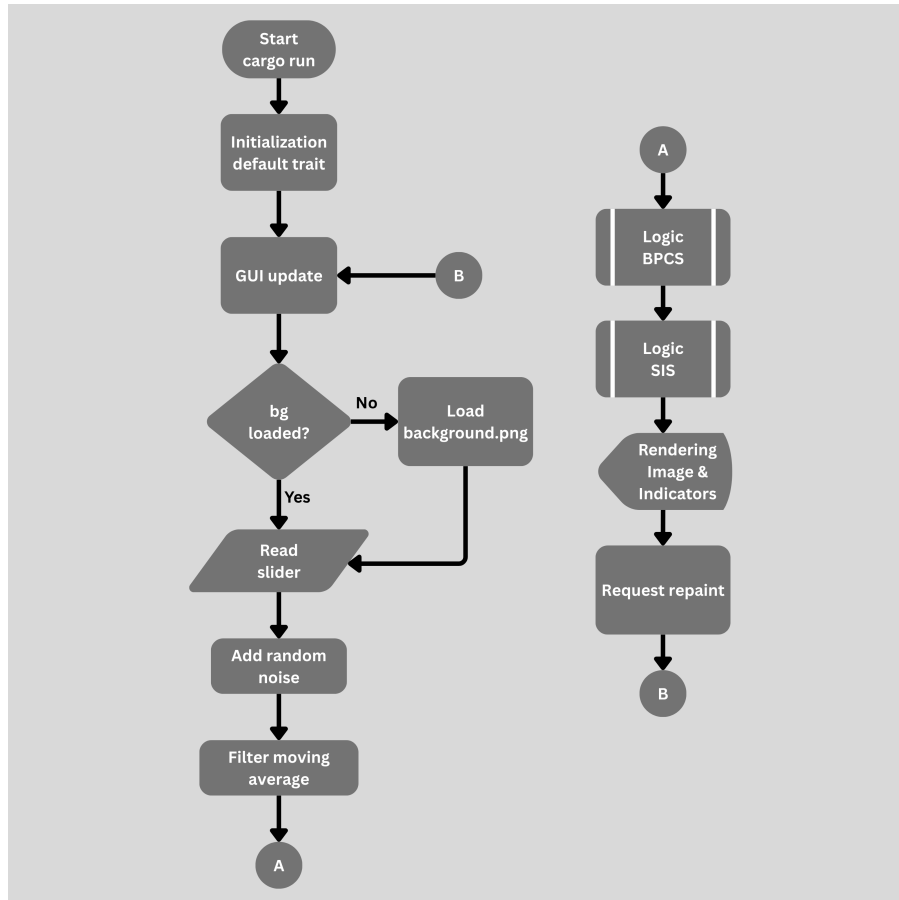


Figure 2: Flowchart Program Utama

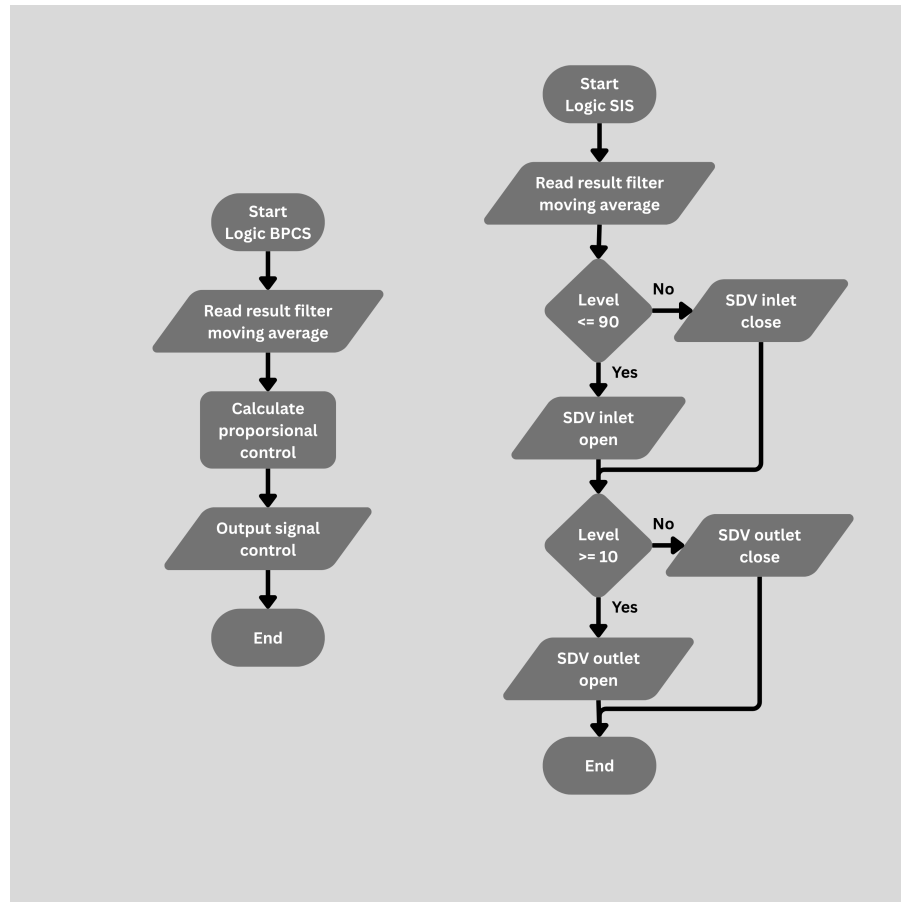


Figure 3: Flowchart Logika BPCS dan SIS

4 Implementasi Program

4.1 Inisialisasi Aplikasi dan Pemuatan Gambar Latar

Salah satu aspek teknis utama project ini adalah pemuatan gambar `background.png` sebagai latar P&ID menggunakan `egui_extras`. Fungsi `egui_extras::install_image_loaders` dipanggil sekali saat inisialisasi untuk mendaftarkan semua *image loader* yang diperlukan, termasuk *decoder* PNG dari crate *image*.

Listing 2: Inisialisasi eframe dan image loader

```

1 fn main() -> eframe::Result<()> {
2     let options = eframe::NativeOptions {
3         viewport: egui::ViewportBuilder::default()
4             .with_title("Simulation_Feed_Water_Tank")
5             .with_inner_size([900.0, 600.0]),
6         ..Default::default()
7     };
8     eframe::run_native(
9         "Simulation_Feed_Water_Tank",
10        options,
11        Box::new(|cc| {
12            // Daftarkan image loader (PNG via crate 'image')
  
```

```

13         egui_extras::install_image_loaders(&cc.egui_ctx);
14         Ok(Box::new(FwtApp::default()))
15     },
16 )
17 }

```

4.2 Struktur Data Utama

struct FwtApp menyimpan seluruh *state* simulasi:

Listing 3: Definisi struct FwtApp

```

1 use rand::Rng;
2
3 struct FwtApp {
4     slider_value:    f32,        // Input operator (0-100%)
5     pv_filtered:     f32,        // Level terfilter (output MA)
6     history:         Vec<f32>,  // Buffer 5 sampel untuk MA
7     setpoint:        f32,        // SP default = 50%
8     inlet_sdv_open:  bool,       // Status Inlet SDV
9     outlet_sdv_open: bool,       // Status Outlet SDV
10    pump_running:     bool,       // Status pompa
11 }

```

4.3 Pemrosesan Sinyal dan Logika Kontrol

Listing 4: Noise filter, BPCS, dan SIS interlock

```

1 // === Noise + Moving Average Filter ===
2 let noise: f32 = rand::thread_rng().gen_range(-1.0..1.0);
3 let noisy = self.slider_value + noise;
4 self.history.push(noisy);
5 if self.history.len() > 5 {
6     self.history.remove(0);
7 }
8 self.pv_filtered =
9     self.history.iter().sum::<f32>() / self.history.len() as
10    f32;
11
12 // === BPCS: Kontrol Proporsional ===
13 let kp: f32 = 1.5;
14 let _cv_bpcs = (kp * (self.setpoint - self.pv_filtered))
15     .clamp(-100.0, 100.0);
16
17 // === SIS: Interlock Independen ===
18 // Interlock 1 - cegah overflow
19 self.inlet_sdv_open = self.pv_filtered < 90.0;
20 // Interlock 2 - cegah dry run pompa
21 self.outlet_sdv_open = self.pv_filtered > 10.0;
22 self.pump_running = self.pv_filtered > 10.0;

```

4.4 Visualisasi Status Komponen

Indikator status SDV dan pompa dirender menggunakan warna berbeda sesuai kondisi aktif atau *interlock*. Tabel 1 merangkum logika pewarnaan tersebut.

Table 1: Logika Pewarnaan Indikator GUI

Komponen	Kondisi	Warna
Inlet SDV	<code>inlet_sdv_open = true</code>	Hijau (Normal)
Inlet SDV	<code>inlet_sdv_open = false</code>	Merah (<i>Interlock</i>)
Outlet SDV	<code>outlet_sdv_open = true</code>	Hijau (Normal)
Outlet SDV	<code>outlet_sdv_open = false</code>	Merah (<i>Interlock</i>)
Pompa	<code>pump_running = true</code>	Biru (Beroperasi)
Pompa	<code>pump_running = false</code>	Abu-abu (<i>Trip</i>)

5 Hasil dan Pembahasan

5.1 Tampilan GUI

Tampilan aplikasi ditunjukkan pada Gambar 1. GUI berhasil merender gambar latar P&ID (`background.png`) menggunakan *egui_extras* sebagai *background* yang informatif. Komponen utama—Inlet SDV, Pump, Level Transmitter (LT), Feed Water Tank, PV indicator, dan Outlet SDV—terletak secara positional di atas gambar latar sehingga membentuk diagram alir proses yang jelas.

5.2 Pengujian Filter *Moving Average*

Filter MA dengan $N = 5$ berhasil meredam fluktuasi akibat *noise* acak $\pm 1\%$ dari crate *rand*. Pergerakan indikator level pada tangki menjadi lebih *smooth* menyerupai respons sensor Level Transmitter industri yang sesungguhnya, dengan *lag steady-state* yang dapat diterima untuk konteks monitoring level.

5.3 Validasi Logika BPCS

BPCS berhasil menghitung sinyal kontrol proporsional yang mendorong PV mendekati *setpoint* 50%. Pada kondisi operasi normal (level antara 10%–90%), kontrol proporsional berfungsi tanpa intervensi SIS, sehingga operator dapat menggerakkan *slider* dan melihat respons korektif sistem secara langsung.

5.4 Validasi Interlock SIS

Dua skenario kritis diuji:

1. **Skenario Overfill:** Slider dinaikkan hingga level mencapai $\geq 90\%$. Indikator Inlet SDV berubah dari hijau menjadi merah, memutus aliran masuk secara paksa tanpa bergantung pada keputusan BPCS.

2. **Skenario Dry Run:** Slider diturunkan hingga level $\leq 10\%$. Outlet SDV menutup dan pompa masuk kondisi *trip* (indikator berubah menjadi abu-abu), melindungi pompa dari kavitasi.

Pada kedua skenario, SIS berhasil mengambil tindakan protektif secara independen, membuktikan pemisahan lapisan kontrol sesuai prinsip IEC 61511.

6 Penutup

6.1 Kesimpulan

1. Aplikasi **Simulation Feed Water Tank** berbasis GUI berhasil dibangun menggunakan Rust dengan *eframe* 0.28.1, menampilkan visualisasi P&ID yang representatif terhadap sistem monitoring industri nyata.
2. Penggunaan *egui_extras* dengan fitur `all_loaders` memungkinkan pemuatan gambar latar P&ID (PNG) yang menjadi fondasi visual antar muka operator.
3. Filter *Moving Average* $N = 5$ berbasis *crate rand* berhasil menstabilkan sinyal sensor simulasi sebelum diteruskan ke lapisan kontrol.
4. Pemisahan lapisan BPCS (kontrol proporsional) dan SIS (*interlock* SDV) terbukti berfungsi secara independen pada kedua skenario kritis yang diuji.

6.2 Saran

1. Menambahkan fitur *trending* berupa grafik level *real-time* menggunakan *crate egui_plot* untuk membantu analisis performa kontroler.
2. Mengimplementasikan kontroler PID lengkap pada BPCS guna menghilangkan *steady-state error* dari kontrol proporsional.
3. Menambahkan fitur *alarm log* berupa catatan waktu kejadian *interlock* SIS untuk keperluan *audit trail* operasional.
4. Mengeksplorasi integrasi protokol Modbus TCP agar simulasi ini dapat dihubungkan dengan perangkat keras instrumen nyata.

References

- [1] R. Fernandez, M. Santos, and J. Cruz, "Development of Low-Cost Water Level Monitoring and Control using PID Controller," in *2024 IEEE 6th International Conference on Control, Robotics and Informatics (IC-CRI)*, pp. 1–6. IEEE, 2024. <https://ieeexplore.ieee.org/document/10425968>

- [2] A. Kumar, P. Singh, and R. Sharma, "Automatic Liquid Level Control of Two Tank System using PLC," in *2024 IEEE International Conference on Industrial Technology (ICIT)*, pp. 1–6. IEEE, 2024. <https://ieeexplore.ieee.org/document/10828622>
- [3] N. A. Nordin, N. A. M. Subha, and A. F. Z. Abidin, "Water Level Control in Boiler System Using Self-tuning Fuzzy PID Controller," in *2023 IEEE 3rd International Conference in Power Engineering Applications (ICPEA)*, pp. 247–252. IEEE, 2023. <https://ieeexplore.ieee.org/document/10237138>
- [4] D. R. Kakani, D. Subendran, and S. J. Singh, "Cascade PID Control Loop Implementation for Liquid Tank Level in LabVIEW PC-Based Control Using Arduino Mega as Data Acquisition," in *2023 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, pp. 1–5. IEEE, 2023. <https://ieeexplore.ieee.org/document/10063911>
- [5] M. Raj, B. Praveena, and R. Velumani, "Virtual Instrumentation under Supervisory Control for Safe Water Level Monitoring," in *2024 International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems (ICICES)*, pp. 1–6. IEEE, 2024. <https://ieeexplore.ieee.org/document/10748479>
- [6] S. N. Pande and A. Sayyad, "Smart System with IoT for Water Quality Monitoring," in *2022 International Conference on Augmented Intelligence and Sustainable Systems (ICAISS)*, pp. 1110–1112. IEEE, 2022. <https://doi.org/10.1109/ICAISS55157.2022.10011108>
- [7] K. Mohan, R. Anand, and S. Balaji, "Smart Water Level Indicator and Control System Using Arduino Uno and Bluetooth," in *2024 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, pp. 1–5. IEEE, 2024. <https://ieeexplore.ieee.org/document/10776937>
- [8] F. Jan, N. Min-Allah, S. Saeed, S. Z. Iqbal, and R. Ahmed, "IoT-Based Solutions to Monitor Water Level, Leakage, and Motor Control for Smart Water Tanks," *Water*, vol. 14, no. 3, p. 309, 2022. <https://doi.org/10.3390/w14030309>
- [9] A. B. Imane, H. R. Mokhtar, and B. Abdelkrim, "Noise Removal in the IMU Sensor Using Exponential Moving Average with Parameter Selection in Remotely Operated Vehicle (ROV)," in *2023 International Conference on Electrical Engineering and Advanced Technology (ICEEAT)*, pp. 1–5. IEEE, 2023. <https://ieeexplore.ieee.org/document/10136259>
- [10] M. R. Islam, S. Azam, B. Shanmugam, and D. Mathur, "An Intelligent IoT and ML-Based Water Leakage Detection System," *IEEE Access*, vol. 11, pp. 123625–123649, 2023. <https://doi.org/10.1109/ACCESS.2023.3329467>

- [11] R. Parasuraman *et al.*, “Human-Centered Automation in Industry 4.0: Design Guidelines for Adaptive HMIs,” *IEEE Transactions on Human-Machine Systems*, vol. 54, no. 1, pp. 15–29, 2024. <https://doi.org/10.1109/THMS.2023.3301425>
- [12] O. Rashad, O. Attallah, and I. Morsi, “A Smart PLC-SCADA Framework for Monitoring Petroleum Products Terminals in Industry 4.0 via Machine Learning,” *Measurement & Control*, vol. 55, no. 9–10, pp. 673–686, 2022. <https://doi.org/10.1177/00202940221103305>
- [13] B. Dwarakanath, P. K. Devi, A. Ranjith Kumar, A. S. M. Metwally, G. A. Ashraf, and B. L. Thamineni, “Smart IoT-Based Water Treatment with a Supervisory Control and Data Acquisition (SCADA) System Process,” *Water Reuse*, vol. 13, no. 3, pp. 411–431, 2023. <https://doi.org/10.2166/wrd.2023.052>
- [14] IEC 61511-1:2016, *Functional Safety – Safety Instrumented Systems for the Process Industry Sector, Part 1: Framework, Definitions, System, Hardware and Application Programming Requirements*. Geneva: International Electrotechnical Commission, 2016.
- [15] E. Ernerfeldt *et al.*, *eframe & equi: An Easy-to-Use Immediate Mode GUI in Rust*, version 0.28, 2024. <https://docs.rs/eframe/>